Scalar TDS Lab

Project Documentation

A local-first dashboard for live Arduino measurements

CHE381 Term Project Team

November 12, 2026

Abstract

Scalar TDS Lab is a responsive web interface for receiving total dissolved solids (TDS) readings from an Arduino, viewing a live experiment, and keeping a local copy of its measurements. The device connected to the Arduino reads the USB serial stream. A lightweight relay shares live readings with other browsers on the same network, while saved experiment logs remain in the browser that collected them. This document describes the implemented features, data flow, serial contract, operating steps, and practical limits of the current foundation.

Keywords: TDS monitoring; Arduino; Web Serial; React; TypeScript; LAN dashboard; Server-Sent Events; local storage; experiment logging; CSV export.

Contents

1 Project overview **2**
1.1 Implemented capabilities . . . 2

2 Architecture and data flow **2**
2.1 Per-device experiments . . . 3
2.2 Local logging versus live sharing . . . 3

3 Arduino interface **3**
3.1 Sensor conversion boundary . . . 3

4 Dashboard behavior **4**
4.1 Five-reading queue . . . 4
4.2 Graphs and network view . . . 4
4.3 Time fields . . . 4
4.4 TDS units . . . 4

5 Saving and exporting **4**

6 Running the project **5**
6.1 Development mode . . . 5
6.2 Production mode . . . 5
6.3 Browser requirements . . . 5

7 Help and practical limits **5**

8 Keywords and component map **6**

1 Project overview

Scalar gives a lab team a shared view of readings without requiring user accounts or a central database. A laptop connected to the Arduino acts as the measurement source. Other devices can open the same app address and follow the current and previous in-memory experiments.

The interface is built with React and TypeScript and bundled by Vite. The backend is a small Node.js HTTP service. It relays readings while the server is running; it does not persist those readings to disk. Saved sessions are written to browser-local storage only when the operator chooses *Save session*.

1.1 Implemented capabilities

- **Live serial input:** the operator selects a USB serial port in a supported browser. The current baud rate is 9600.
- **TDS overview:** the dashboard displays the latest TDS result, optional temperature, sample count, and experiment duration.
- **TDS unit selector:** values are shown as ppm or mg/L. The selector changes the displayed unit label; it does not apply a numeric conversion.
- **Five-reading queue:** the table keeps the latest five samples visible. New samples are appended at the bottom and the oldest visible row rolls off the top.
- **Live graphs:** a line graph follows the current local experiment. Network experiments have their own graphs and expandable reading tables.
- **LAN sharing:** the host relay broadcasts readings and experiment history to connected viewers using Server-Sent Events (SSE).
- **Local saving and export:** the collecting browser can save its experiment and download CSV or tab-delimited TXT files.
- **In-app guide:** a Help section describes network setup, serial connection, experiment viewing, and saving/exporting.

2 Architecture and data flow

Figure 1 shows the path from the Arduino to the viewers. Data travels to the relay for temporary sharing. A separate local-storage path is taken only when the operator saves a session.

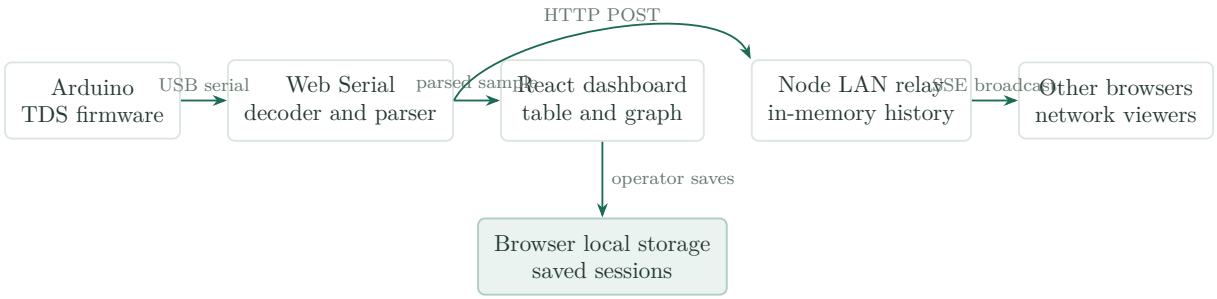


Figure 1: Scalar data flow. The relay shares readings only while its process is running; the local save path is explicit and device-specific.

2.1 Per-device experiments

Each browser tab receives a temporary source ID. When a tab publishes readings, the relay groups them under that source and experiment ID. If multiple devices connect Arduino boards, each one appears as a separate network experiment. A viewer can inspect each experiment's graph and full in-memory table.

2.2 Local logging versus live sharing

The relay keeps shared readings in process memory. Restarting the server clears this shared history. The local operator can separately save a session to that browser's local storage; clearing the browser's site data removes that local copy. Network viewers can inspect the live history but do not automatically receive or save a copy. CSV or TXT export is the supported way to transfer saved data.

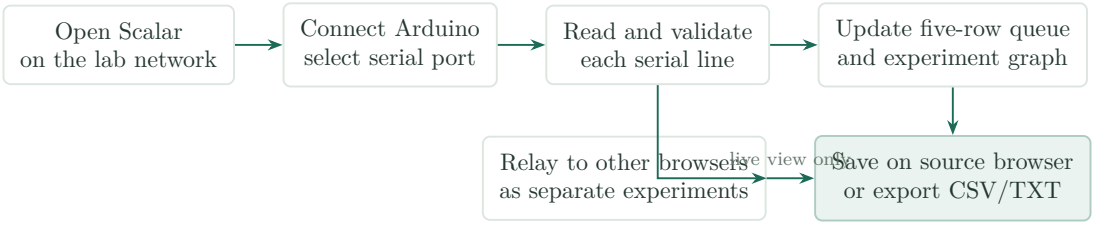


Figure 2: Typical operator workflow. A source browser collects and optionally saves; other browsers follow the shared stream.

3 Arduino interface

The browser opens the selected port at 9600 baud. The current version expects one UTF-8 CSV row per sample, ending with a newline. The protocol is versioned so future firmware can evolve without guessing column meanings.

Listing 1: Scalar serial protocol, version 1

```
SCALAR,1,1240,245.50,24.8
```

Column	Meaning
`SCALAR`	Protocol marker.
`1`	Protocol version.
`1240`	Arduino elapsed time from `millis()`, in milliseconds.
`245.50`	Calibrated TDS concentration, in ppm.
`24.8`	Optional temperature, in degrees Celsius.

The browser attaches a local receive timestamp to each parsed sample as well. This makes the sample usable when viewed on a different device and preserves the Arduino's uptime as a separate timing field. The parser also accepts early prototype formats, but new firmware should use the versioned row above.

3.1 Sensor conversion boundary

The serial application reads and displays a TDS value; it does not infer TDS from the probe circuit. The actual Arduino firmware must calculate a calibrated TDS value and send it in ppm.

That conversion depends on the sensor, circuit, calibration solution, and project method, which are not specified by the web application.

The included `arduino/scalar_serial_test.ino` is a synthetic transport generator. It produces changing example values at one-second intervals. It is useful for checking the serial path and interface, but it is not a TDS instrument and its measurements must not be treated as experimental results.

4 Dashboard behavior

4.1 Five-reading queue

The local table displays the five most recent samples in chronological order. When the sixth sample arrives, the oldest visible sample rolls off the table. The complete active reading list remains in session state and is used by the chart, network relay, save operation, and export. The queue is therefore a compact live window, not a limit on the experiment length.

4.2 Graphs and network view

The local graph plots readings from the current experiment. The network section renders a separate latest value, graph, and expandable full table for each source experiment received from the relay. Network values are displayed using the current browser's selected TDS unit.

4.3 Time fields

Each row has two useful times:

- **Local receive time:** the browser's local clock time when it parsed the incoming line.
- **Arduino uptime:** the board's elapsed milliseconds, displayed in seconds for easier reading.

Arduino uptime resets when the board restarts. Local receive time is more useful for comparing what different viewers saw; uptime is useful for understanding order and spacing within one board's run.

4.4 TDS units

The serial value and saved numeric value are in ppm. The selector offers ppm and mg/L. These units represent mass fraction and mass per volume, respectively; they are approximately equivalent for dilute water samples. Scalar keeps the numeric value unchanged when the display label changes. This is a presentation choice, not a density-aware conversion. The USGS discusses this approximation for dissolved-solids concentrations below roughly 7,000 mg/L[1].

5 Saving and exporting

1. Connect the Arduino and collect readings.
2. Use the graph and recent-reading table to follow the run.
3. Select *Save session* on the collecting browser to add the full run to that browser's local experiment list.
4. Choose CSV or TXT from the export menu to download the current or saved data. The TXT option uses tab delimiters for straightforward reading in a text editor or spreadsheet.

[1] USGS, *Dissolved Constituents Including Selenium in Waters*, Water-Resources Investigations Report 85–4220, conversion factors, available at https://pubs.usgs.gov/wri/1985/4220/report.pdf.

Exported rows contain the experiment name, local timestamp, Arduino elapsed time, TDS in ppm, and temperature when present. The display unit selection does not change the exported ppm data, which remains consistent across browsers.

6 Running the project

6.1 Development mode

From the project directory, install dependencies and start Vite:

```
npm install
npm run dev
```

Open the printed local URL on the host computer. To view the dashboard on another device on the same Wi-Fi, use the host computer's LAN IP address with port 5173. The host must remain running to relay live readings.

6.2 Production mode

Build the client and start the Node server:

```
npm run build
npm start
```

The production server serves the `dist` directory and exposes the live feed on port 4174 by default. The `PORT` environment variable can override the port. Production deployments that cross a public or untrusted network should use HTTPS and appropriate network restrictions.

6.3 Browser requirements

The dashboard is responsive and can be viewed on phones, tablets, and computers. USB serial access is separate from page viewing: it requires a compatible browser and the Arduino plugged into the device that selects the port. Web Serial is supported mainly in desktop Chromium browsers and requires a secure context, such as localhost or HTTPS.

7 Help and practical limits

The in-app Help section summarizes setup and the serial contract. These points are useful during the first lab run:

- Confirm the host computer and viewers are on the same local network.
- Connect the Arduino from the computer that has the USB cable; viewers do not need a USB port.
- Confirm the board uses 9600 baud and emits one newline-terminated row per sample.
- Confirm the firmware sends a calibrated TDS value in ppm, not raw ADC counts or conductivity, unless the firmware performs the intended conversion.
- Save data from the collecting browser before closing the page or clearing site data.

There is no login or access-control layer. Anyone who can reach the app on the LAN can view the feed. The relay is intended for a trusted lab network, not for an open public deployment. The server's in-memory history is temporary, and the synthetic Arduino sketch does not validate the real sensor conversion.

8 Keywords and component map

Keywords: total dissolved solids; TDS; Arduino; serial telemetry; Web Serial API; React; TypeScript; Vite; Node.js; Server-Sent Events; LAN collaboration; local storage; experiment graph; ppm; mg/L; CSV; TXT.

Project component	**Responsibility**
`src/App.tsx`	Dashboard, local session flow, shared experiment panels, and Help content.
`src/services/serialDevice.ts`	Browser serial port and line reading.
`src/services/measurements.ts`	Versioned serial parsing and TDS display-unit behavior.
`src/services/sharedFeed.ts`	Client-side live feed, publisher, and source identity.
`server/liveFeed.mjs`	In-memory experiment collection and SSE broadcast.
`server/index.mjs`	Production static-file and API server.
`src/services/sessionRepository.ts`	Browser-local saved-session persistence.
`arduino/scalar_serial_test.ino`	Synthetic serial test data for interface checks.